CIPFP BATOI

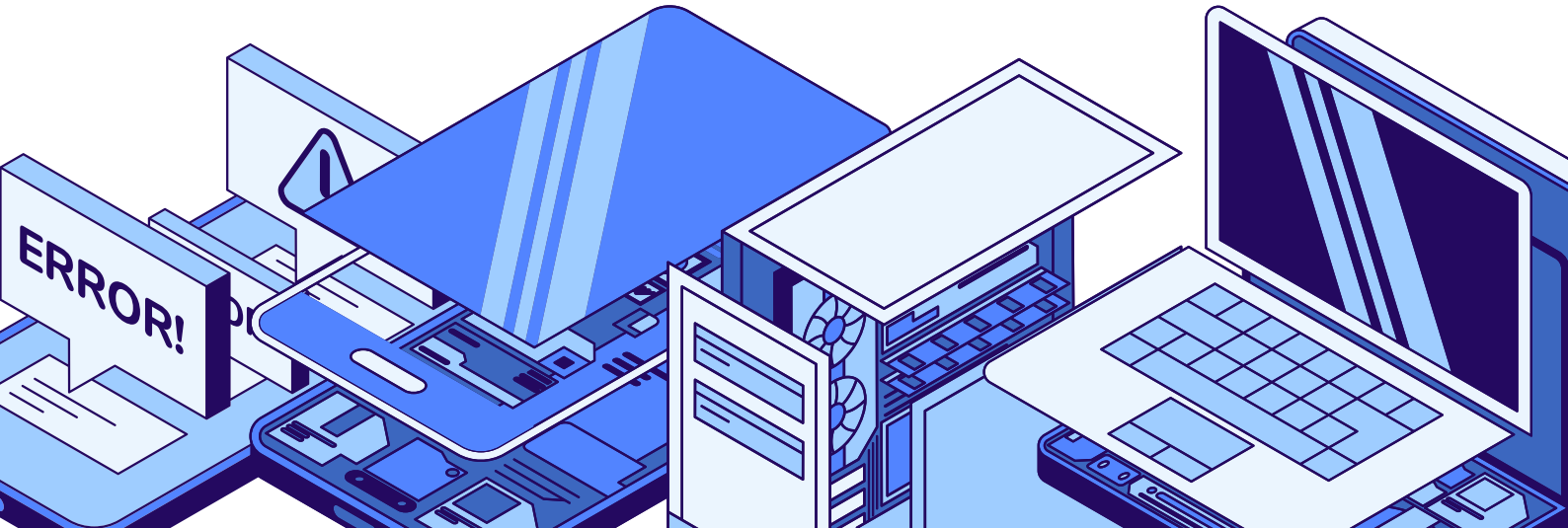
gestor de tareas

TASKNEST

Desarrollo de Aplicaciones Web

17-12-2025

Alumna:
Andrea Pérez Carbonell
Tutor:
Iván Martos Gázquez



Nombre del proyecto
TaskNest

Autor
Andrea Pérez Carbonell

Ciclo Formativo de Grado Superior
Desarrollo de aplicaciones web

Fecha
17/12/2025

Nombre del centro educativo
CIPFP Batoi

Tutor individual
Iván Martos Gázquez

Agradecimientos

En primera instancia quiero agradecer a todas las personas que han confiado, me han apoyado y dado ánimos en la realización de este proyecto. Sobre todo, en cuanto a mi familia, ya que gracias a ella he podido tener la idea de crear una aplicación de gestión de tareas de casa para poder facilitar está gestión en distintos hogares.

También me gustaría agradecer a mi tutor del proyecto, por su apoyo y sugerencias para mejorar el proyecto, además de la disponibilidad para resolver dudas.

Índice

1. Introducción.... 4
1.1. Motivos por los que se ha elegido el proyecto.... 4
1.2. Descripción y objetivos generales del proyecto.... 4
1.3 Posibles aplicaciones prácticas del proyecto.... 5
2. Fundamentos generales, teóricos y prácticos.... 6
3. Fases de Implementación técnica.... 9
3.1. Preparación del proyecto.... 9
3.2. Arquitectura del proyecto.... 11
3.3. Desarrollo del proyecto.... 12
3.4. Diseño e implementación de la base de datos.... 13
3.5. Funcionalidades de la aplicación.... 17
3.6. Recursos utilizados.... 23
4. Estudio del coste económico y organizativo.... 25
5. Comparación con la situación actual y soluciones alternativas.... 26
6. Conclusiones.... 27
6.1. Resultados obtenidos.... 27
6.2. Puntos pendientes.... 27
6.3. Tiempo dedicado, dificultad del proyecto en general y problemas.... 28
6.4. Valoración personal y auto evaluación.... 29
6.5. Consejos más destacables.... 30
7. Necesidades y sugerencias de formación.... 30
8. Bibliografia.... 31

1. Introducción

1.1. Motivos por los que se ha elegido el proyecto.

Tras mucho tiempo de reflexión sobre la temática del proyecto, en última instancia se ha decidido hacer un proyecto que cubra una necesidad personal, como es el caso de una aplicación de gestión del hogar. A raíz de cubrir una necesidad, esta idea fue cobrando forma y se pensó que tecnologías serían más adecuadas para llevarla a cabo y que en un futuro se pudiera escalar como aplicación móvil. Es por eso que se decidió dividir la aplicación en dos partes, una dedicada a la conexión con la API y la otra que fuera la lógica funcional de la aplicación. Todo esto al querer que fuera escalable se decantó por usar docker para contener estas dos partes de la aplicación.

Por otro lado, tras haber empezado las prácticas en la empresa del grado superior, no se decidió cambiar esta idea, si no mejorarla. Ya que, al tener un ejemplo real de una aplicación web, se ha podido comparar y aprender qué funciones son necesarias para facilitar la usabilidad y la experiencia de los usuarios al utilizar una aplicación. Esta experiencia ha tenido una gran influencia en el diseño y funcionamiento de la aplicación.

En conclusión, los motivos por los cuáles se ha decidido realizar una aplicación web qué se encargue de gestionar tareas del hogar, son la necesidad de una ayuda para gestionar un inventario, además de tener en un mismo sitio guardadas las tareas, eventos y una lista de la compra. Ya que la idea es que se pueda utilizar en una tablet y poder actualizarlo en casa desde la cocina o cualquier estancia de la casa.

1.2. Descripción y objetivos generales del proyecto.

En primer lugar, este proyecto se trata de una aplicación web destinada a la gestión de tareas de un hogar, se ha decidido que el nombre sea TaskNest, ya que significa nido de tareas en referencia al hogar. Pero no es tan solo un gestor de tareas, también permite al usuario modificar y añadir productos a la despensa, para que pueda añadirlos a la lista de la compra si el stock de estos es bajo. Además de tener un apartado de calendario donde se puedan visualizar los eventos que se hayan creado como pueda ser una cena, un cumpleaños o una fecha importante.

En un primer momento, con la realización de esta aplicación se pretendía crear funcionalidades básicas en cuanto a la gestión de tareas, ya que el objetivo era implementar la funcionalidad de escanear los productos a través del código de barras con un escáner, en este caso una cámara, ya sea la del portátil como la cámara trasera del teléfono móvil. Por tanto, el objetivo ha sido explorar la mecánica

de escanear los productos de manera que se pueda compatibilizar con el proyecto en sí, es decir adaptarlo a la funcionalidad de API que tienen Php y Laravel 11, conectándolo con las vistas y los servicios de Vue 3.

La elección del proyecto y de esta funcionalidad de escáner como objetivo general del proyecto, ha sido así para explorar y afianzar los conocimientos sobre cómo hacer de una API un recurso funcional y optimizado, para poder aprender la lógica de empezar un negocio a través de una aplicación web, por la razón de que este proyecto aún puede seguir adaptando nuevas funcionalidades y ajustes, como poder ser también una aplicación móvil escalable y ser usada por usuarios reales. Además, de que ha servido para consolidar los conocimientos sobre lenguaje de servidor como es php y poder aplicar ese conocimiento en las prácticas.

En conclusión, se ha invertido mucho tiempo del proyecto en la arquitectura y la funcionalidad general de la aplicación, para tener una base robusta y escalable.

1.3 Posibles aplicaciones prácticas del proyecto.

Como bien se ha explicado en el apartado anterior de este primer punto, las posibles aplicaciones prácticas que puede tener el proyecto, sería hacer una versión de la parte del frontend nativa para móvil, es decir todo aquello que está funcionando en Vue que pueda funcionar en dispositivos móviles. Además de añadirla a las diferentes plataformas de descarga de aplicaciones móviles como google play store, app store, etc.

Esto permitiría un acceso más cercano a futuros usuarios, para poder utilizar la aplicación web en cualquier momento, por ejemplo una de las aplicaciones más útiles sería cuando una persona va a comprar en el supermercado y requiere tener la lista de la compra a mano, además de poder actualizarla en ese mismo instante eliminando y añadiendo productos que se le hayan podido olvidar o que en el momento de realizar la lista no se han tenido en cuenta. A parte de eso, cabe destacar que otra gran utilidad de esta aplicación sería el uso del escáner de código de barras de los productos que añadimos en nuestro carro a la hora de comprar, así con un simple gesto podemos añadir los productos a la despensa actualizando su stock y a su vez editar la lista de la compra.

Aún así, como se explicará más adelante aún se pueden implementar más funcionalidades y utilidades para aquellos hogares que tengan sistemas inteligentes, como persianas automáticas, luces conectadas a dispositivos como Alexa… En estos casos se podría tener una vista para controlar estos dispositivos a través de domótica, pudiendo controlarlos a través de la aplicación de TaskNest.

2. Fundamentos generales, teóricos y prácticos.

Este proyecto se ha centrado más en los conocimientos obtenidos durante el segundo curso del grado superior de desarrollo de aplicaciones web, pero eso no significa que no se haya utilizado lo aprendido durante el primer curso. A continuación se va a explicar qué asignaturas han tenido una mayor presencia en la realización de este, dividiéndolas por cursos.

En cuanto al primer curso, las asignaturas que han tenido más relevancia son las siguientes:

Programación
Esta es la asignatura base de todo el grado superior, ya que en ella es donde se aprenden las bases de la programación como son los métodos, las clases, herencia y la programación orientada a objetos (POO). Es una asignatura necesaria de entender, para poder realizar cualquier proyecto o trabajo que implique un desarrollo.

Lenguajes de Marcas
En esta asignatura es donde se empieza a trabajar con Javascript, html y css orientado al diseño de una página web, resulta una asignatura que también establece las bases de diseño y permite entender los diferentes tipos de archivos y formatos como pueden ser los archivos json, xml, yml, etc. Comprender la estructura de este tipo de archivos resulta muy necesario para poder trabajar con peticiones a un servidor como se ve en profundidad en segundo.

Entornos de Desarrollo
En el ámbito de la programación resulta muy necesario tener un control de versiones, sobre todo cuando estamos trabajando con un entorno de desarrollo integrado (IDE), ya que este tipo de entornos son los que facilitan poder escribir código y detectar errores en tiempo de ejecución con mayor facilidad, ya que se componen de herramientas o expansiones para poder detectar errores de sintaxis o malformación de código no identado que según el lenguaje en el que estemos trabajando pueda afectar a la compilación del código. Es por eso que está asignatura es tan necesaria para poder entender qué opciones se tienen a la hora de empezar un nuevo proyecto dependiendo de la envergadura y el lenguaje que se vaya a utilizar. Aunque sirve de base para empezar, es cierto que en este ámbito es donde más cambios se encuentran según pasan los años, ya que por ejemplo ahora mismo está muy extendido el uso de Visual Studio Code incluso en empresas grandes, gracias a las funcionalidades y adaptabilidad que tiene, además de la conexión de github para poder hacer el control de versiones de manera casi automática.

En el caso de este proyecto, la aplicación práctica ha sido utilizar github con Visual Studio Code para tener un control de versiones actualizado en todo momento y poder compartir el proyecto con el tutor, además de poder poner un tipo de licencia, en este caso el tipo de licencia es la de tipo MIT, es decir la más básica.

En definitiva, se trata de una asignatura que aporta muchas herramientas para los desarrolladores, así como entender el flujo que tiene un proyecto de desarrollo y qué opciones se tienen.

Bases de Datos

Esta es una de las asignaturas más importantes dentro del mundo de la programación junto a la asignatura de programación. La razón de su importancia, es que sin una base de datos no se puede trabajar ni desarrollar ningún proyecto, ya que las clases y la programación orientada a objetos está intrínsecamente relacionada con la realización de un buen diseño de base de datos. Por tanto, en esta asignatura se aprende todo lo relacionado con el diseño de las bases de datos relacionales como también los gestores de bases de datos que podemos utilizar, para tener una conexión con la base de datos y poder hacer consultas a esta de manera más sencilla y rápida, sobre todo cuando se está realizando peticiones al servidor y no sabemos qué datos queremos obtener a través de la petición, una buena manera de comprenderlo es hacer una consulta a la base de datos para entender qué datos estamos obteniendo y si son los que queremos.

En este proyecto el diseño de base de datos ha sido imprescindible, ya que es lo primero que se realizó, junto a las migraciones y los seeders para poder rellenar la base de datos con datos ficticios pero consistentes. Concretamente, se ha utilizado el gestor PhpMyAdmin para la gestión de la base de datos basada en MySql. Más adelante se expondrán las diferentes tablas y relaciones de la base de datos creada, aunque hay algunas de estas tablas que se crean automáticamente para la gestión de sesiones y que son necesarias para la configuración base del proyecto.

Por otra parte, en cuanto al segundo curso. las asignaturas que han tenido más relevancia son las siguientes:

Desarrollo de Aplicaciones en Entorno Servidor

Esta asignatura es un pilar dentro de la programación, sobre todo en el desarrollo de aplicaciones web, ya que en ella se aprende a cómo desarrollar una aplicación que parte del cliente y llega al servidor para obtener los datos necesarios para poder hacer operaciones o peticiones de tipo CRUD (crear, listar, actualizar y eliminar). Además de esto, se enseña el funcionamiento de php con docker y como preparar el entorno para su instalación. Por último, otro aspecto a destacar de esta asignatura es el uso de frameworks para trabajar de manera más eficiente con lenguajes como php, uno de los frameworks vistos es laravel. Y es justo la tecnología que se ha usado en el desarrollo de este proyecto.

Por tanto, esta asignatura ha sido un pilar para poder llevar a cabo el proyecto. A parte, se ha decidido usar esta tecnología, ya que hay muchas páginas web que en la actualidad siguen usando php como lenguaje de servidor para servir sus páginas/aplicaciones web, es por eso que para afianzar el conocimiento de este lenguaje se ha utilizado php. Como también Laravel, ya que con la nueva versión Laravel 11 han incluido mejoras y cambios que no pasan desapercibidos en el mundo de la informática. De hecho una de las complicaciones que hablaremos más adelante de este proyecto ha sido usar la última versión de Laravel.

Desarrollo de Aplicaciones en Entorno Cliente

Por otra parte, está asignatura es la otra cara de las aplicaciones web, ya que es en la que se aprende a desarrollar lo que conocemos como front-end, que es básicamente la vista que tiene el cliente de nuestra aplicación, es decir la parte más visual que a través de peticiones le pide al servidor los datos almacenados en la base de datos y los muestra de manera visual en nuestra web. Es una asignatura completa y muy útil, ya que además de aprender cómo funciona una SPA (Single Page Application), se enseña mucho más sobre javascript y todo lo que este lenguaje implica en la funcionalidad de una web.

En este caso, está asignatura ha servido para poder hacer todo el front-end de la aplicación, además de funcionalidades extras que no tienen que ver con el servidor como son botones de dark mode o el botón de hamburguer, son botones que están muy a la orden del día en casi cualquier página o aplicación y que cualquier usuario a día de hoy conoce.

Despliegue de Aplicaciones Web

En cuanto a la asignatura de despliegue, considero que es una de las más complicadas del curso, ya que requiere tener un conocimiento más avanzado de la terminal para poder establecer el servidor de un proyecto así como desplegarlo en última instancia. Es una asignatura muy completa en cuanto a conocimientos, y se trata de un aprendizaje muy importante en el futuro de una persona que se dedica al desarrollo de aplicaciones web.

Como no podía faltar, en este proyecto ha sido un pilar para poder contenerizar tanto el front-end (desarrollo en entorno cliente) como el backend (desarrollo en entorno servidor) usando docker. Además de los comandos usados en al terminal de linux mint para poder instalar las librerías y dependencias que se han usado en la aplicación de TaskNest.

Diseño de Interfaces Web

Por último en cuanto a las asignaturas que han tenido un papel fundamental en el desarrollo de esta aplicación web, está el diseño de interfaces web. En esta asignatura se indaga más sobre los conceptos en los que se trabajan en la

asignatura de lenguaje de marcas, es decir se aprende a cómo diseñar una página web siguiendo unos parámetros de estilos para hacer que la aplicación sea más cercana y fácil de usar para los futuros usuarios de esta. Como también se aprende a trabajar los estilos de manera más profunda usando clases de bootstrap y otros formatos de estilos como mixins.

En cuanto a su utilidad en este proyecto, ha sido muy importante ya que tanto para los estilos de la web, como los formularios y botones se ha utilizado bootstrap, tailwind CSS y CSS para configurar el diseño y los estilos de la aplicación web.

3. Fases de Implementación técnica

En este apartado, se va a exponer cómo ha sido el desarrollo del proyecto, viendo cómo está organizada su arquitectura de manera resumida, además de las tecnologías y librerías que se han utilizado para llevar a cabo la realización de ciertas funcionalidades que más adelante se explicarán. Como también se expondrá la estructura del proyecto y de la base de datos, además de su diseño y composición.

3.1. Preparación del proyecto.

En este apartado se va a explicar cómo ha sido la organización del proyecto en general, ya que antes de ponerse a desarrollar un proyecto nuevo desde cero, hay que tener muy claro qué tecnologías y recursos se van a utilizar, además de cómo va a estar organizada su estructura.

En este caso al ser una aplicación que divide el frontend (la capa del cliente) y el backend (la capa del servidor). Se tuvo claro desde un principio que al trabajar de manera separada con Laravel y Vue se necesita una tecnología como docker para poder contenerizar el proyecto. Además de esta razón, docker tiene diversas ventajas que se han podido aplicar a este proyecto como es desplegar la aplicación entera sin tener que hacer docker compose up -d y npm run dev, si no que simplemente con el comando docker compose up -d ambas partes del proyecto funcionaran a la vez. Otras de las razones para utilizar docker, es que esta aplicación está pensada para ser escalable en un futuro, por lo que es importante y necesario que se pueda trabajar de manera dividida entre el frontend y el backend. Teniendo en cuenta que si la aplicación se hace más grande y se necesita de un equipo de desarrolladores, no habrá problemas a la hora de trabajar en distintas partes de esta.

Otro aspecto importante en la preparación, es la gestión del tiempo ya que al tener un tiempo límite hay que saber organizar la carga de trabajo y los objetivos a conseguir. En este caso el proyecto empezó en Octubre y ha finalizado en Diciembre, por lo que han sido tres meses de realización y puesta en marcha. A la

parte que más horas se ha dedicado es a la preparación y configuración del backend, así como la base de datos, ya que la parte de Vue una vez estaba hecha la lógica y la conexión con la API era trabajar con el diseño y que funciones se necesitaban implementar en las diferentes vistas.

Por último, para poder preparar este proyecto se ha necesitado realizar diversas cosas como se puede apreciar a continuación:

1. Crear un nuevo repositorio en GitHub para tener un control de las versiones del código, así como un seguimiento de los commits. Cabe destacar, que contiene un readme con lo necesario para descargar e implementar el proyecto, así como la licencia MIT, es una de las licencias más básicas que ofrece github, la cual permite compartir el proyecto. Y como es un proyecto que va a tener una continuidad en su desarrollo se ha optado por este tipo de licencia más permisiva.

2. Clonar el repositorio en una carpeta local del disco duro SSD utilizado durante el superior con un sistema operativo de Linux Mint. Además de crear la rama develop para trabajar en ella de manera segura sin afectar a producción, que en este caso se trata de la rama main.

3. En este caso ya se tenían instalados los siguientes programas y recursos, por lo que no ha sido necesaria su instalación: Visual Studio Code, NPM y Docker. Pero, si que se ha necesitado instalar en el bash de la parte backend php junto a la nueva versión de Laravel, así como PhpMyAdmin como gestor de la base de datos. Y, por lo que corresponde al frontend de esta aplicación ha sido necesaria la instalación de Vue 3 con javaScript. Más adelante se hará hincapié en los recursos y librerías que se han ido instalando y necesitando.
4. Una vez se ha tenido todo lo necesario para empezar el proyecto, se ha realizado el diseño de la base de datos relacional, basada en MySql, ya que php trabaja muy bien con el lenguaje de bases de datos como es MySql.

5. Una vez se ha tenido el diseño de la base de datos, se definen los modelos y controladores que va a tener la aplicación, así como las migraciones y los seeders, para tener datos y poder trabajar con datos más reales.

Una vez realizados todos estos pasos, se ha dedicado la mayor parte del tiempo del proyecto al desarrollo de este. Incorporando la parte del diseño web y las clases de bootstrap y Tailwind Css más adelante.

3.2. Arquitectura del proyecto.

Por otro lado, una vez explicada la importancia que ha tenido la elección de la arquitectura de carpetas del proyecto y cómo se relacionan entre sí, a continuación se adjunta una captura de la estructura final de la aplicación.

```
taskNest/
├── backend-laravel/
│   ├── (código Laravel)
├── frontend-vue/
│   ├── (código Vue)
└── docker/
    ├── docker-compose.yml
    ├── php/
    │   └── Dockerfile
    ├── nginx/
    │   └── default.conf
    └── node/
        └── Dockerfile
```

Para poder entender esta arquitectura como se desarrolla desde la perspectiva del desarrollo de aplicaciones web, se va a explicar cada apartado de manera breve.

En cuanto al contenido de la carpeta de backend-laravel, es donde se encuentra toda la estructura base de la aplicación respecto a la API y conexión con la base de datos, ya que en esta se ubican los modelos de las y las migraciones correspondientes a las tablas que tiene la base de datos, así como también los controladores API que tienen las funciones CRUD para poder obtener la información de las variables de las distintas tablas de la base de datos.
Otro aspecto importante, y más trabajando con laravel son las rutas api y web. Como se ha mencionado antes, se ha realizado este apartado con Laravel 11 es decir que ahora el proyecto no viene con un fichero de rutas api, por lo que en la configuración bootstrap/app se ha tenido que añadir la línea del directorio correspondiente a las rutas api. En otras versiones de Laravel esto no era necesario ya que se configuraban los ficheros de rutas de manera independiente.

No obstante, en la carpeta de frontend-vue, es donde se encuentra todo lo relacionado con los ficheros de Vue 3, es decir los componentes, las vistas, los servicios, así como las rutas index.js y Axios, uno de los aspectos más importantes del frontend, ya que es una librería que sirve para hacer peticiones HTTP hacia una API, en este caso la conexión con laravel.

Por último tenemos la carpeta de Docker que es donde se ubica toda la configuración de este incluyendo los datos de conexión de la base de datos, así como la configuración del servicio nginx. En el archivo de docker-compose.yml, se ha añadido la conexión del puerto que utiliza Vue, para que con solo ejecutar el

comando docker compose up -d, la parte del frontend también se despliegue. Cabe destacar que el proyecto se ha realizado desde localhost:8080. No se ha desplegado la aplicación en un dominio real, ya que se ha considerado una estructura básica en cuanto a lo que puede llegar a ser la aplicación en sí.

3.3. Desarrollo del proyecto.

En este apartado explicaremos las tecnologías elegidas para cada parte del desarrollo de la aplicación web de tareas de casa (TaskNest).

Con respecto a la parte del **Servidor Web**, es la parte esencial de la aplicación, concretamente, funciona como una API Rest para gestionar las peticiones que ocurren desde la parte del cliente. Como funciona como una API, se ha elegido desarrollar este apartado con PHP 8.3.6, es la versión más actual y compatible con laravel 11 que es el framework elegido para trabajar con php. Se ha decidido trabajar con este framework, ya que la curva de aprendizaje es más baja que en otros como Symfony. Además de estar adaptado visualmente al MVC (Modelo Vista Controlador) en cuanto a la distribución de los ficheros. Asimismo tiene fácil configuración para trabajar con Vue 3.

A parte de la facilidad de uso de este framework, aporta funcionalidades esenciales a una aplicación como la herramienta de Laravel Sanctum, en este proyecto se ha intentado entender cómo funcionan estas herramientas, aunque no se hayan podido aplicar del todo debido a la falta de tiempo y errores durante su configuración. No obstante sí que se ha hecho uso de recursos como **php artisan tinker** para poder comprobar la dirección de las rutas y el funcionamiento de los modelos, en los casos en los que no daba error ni de servidor ni en consola, pero los resultados no eran los que se pretendían.

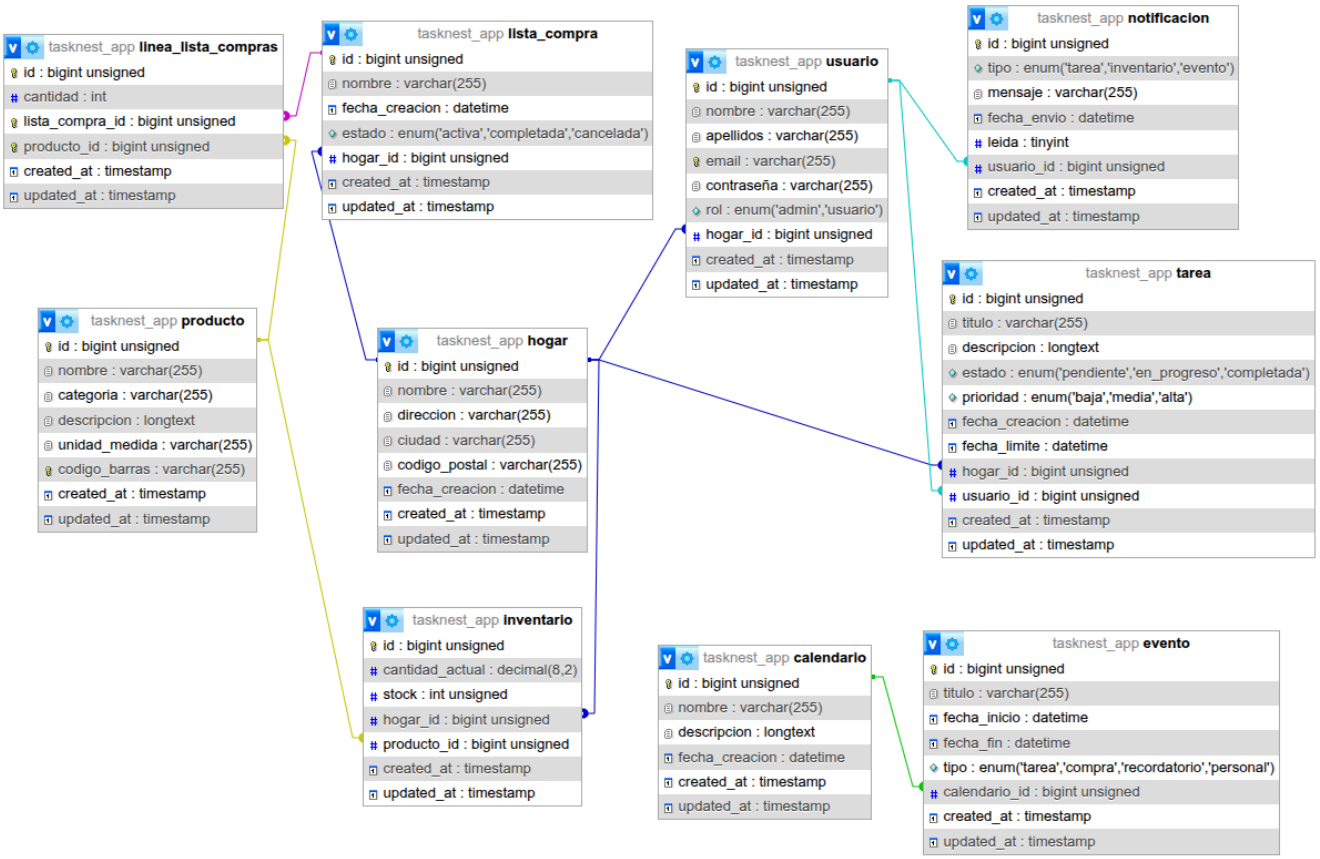
En cuanto a la parte del **Cliente Web**, se ha decido utilizar Vue 3 con JavaScript en vez de utilizarlo con TypeScript, debido a que al haber estudiado Js se ha podido tener en cuenta la arquitectura del código y aplicar la lógica de las peticiones a la API usando Axios. Por la razón de que TypeScript hubiera supuesto una complicación añadida a la hora del desarrollo de la aplicación y en este caso el objetivo se ha centrado en desarrollar una aplicación web de tareas de casa incluyendo funcionalidades como calendario, eventos y un escáner de productos, que más adelante se explicará su funcionalidad.

Por último, en referencia a la **base de datos** se ha decidido utilizar PhpMyAdmin debido a su compatibilidad con docker, ya que funciona como un contenedor más y permite inicializar el puerto directamente desde docker, lo que facilita la integración del proyecto sin necesidad de instalaciones añadidas, así como también el crear bases de datos relacionales usando MySql. Sin embargo, se hubiera podido utilizar un gestor de bases de datos como Dbeaver, pero como en este proyecto al utilizar

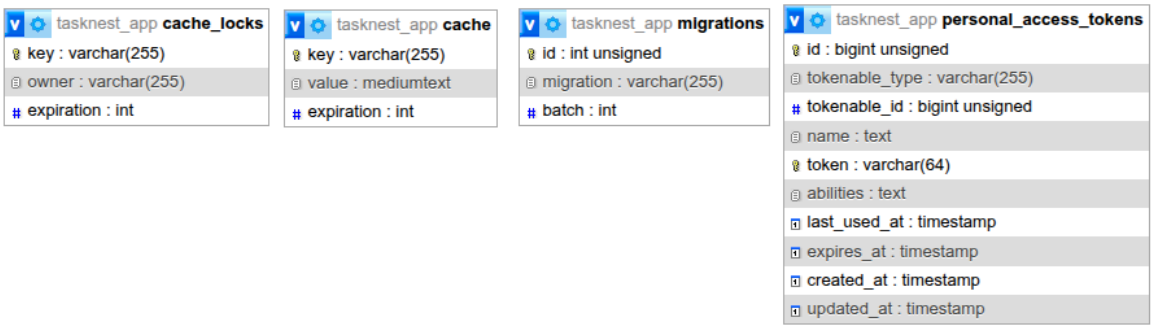
laravel, las consultas se pueden utilizar con eloquent, no es estrictamente necesario utilizar MySql para hacer consultas a la base de datos desde los controladores. Además otro aspecto importante a destacar es que en PhpMyAdmin la ejecución de las migraciones y los seeders se actualizan inmediatamente, mostrando en tiempo real si ha habido algún tipo de problema.

3.4. Diseño e implementación de la base de datos.

El apartado siguiente corresponde al diseño de la base de datos, para poder visualizar las relaciones, se ha adjuntado una imagen del diagrama E/R desde el propio gestor de PhpMyAdmin, en la siguiente imagen se puede observar las tablas principales de la aplicación, junto con sus atributos, claves foráneas y claves primarias que hacen posible la relación con otras tablas.



Por otro lado, se ha adjuntado también una imagen de las tablas básicas de la configuración de la base de datos y de la aplicación, com son las tablas donde se guarda la caché, las migraciones y los tokens personales, para poder acceder a la aplicación de manera segura sin inyecciones de código.



A continuación, se va realizar una pequeña explicación de los campos de cada tabla de la base de datos.

HOGAR

- Id, se trata de la clave primaria.
- Nombre, es el nombre del hogar.
- Dirección, es la dirección donde se ubica el hogar.
- Ciudad, es la ciudad donde se ubica este hogar.
- Código_postal, es el código de la dirección del hogar
- Fecha_creación, es la fecha en la que se crea este registro.
- Created_at, son los timestamps que se definen en la migración. En este caso cuando se crea un registro.
- Updated_at, son los timestamps que se definen en la migración. En este caso cuando se edita el registro.

USUARIO

- Id, se trata de la clave primaria.
- Nombre, es el nombre del usuario.
- Apellidos, son ambos apellidos del usuario.
- Email, campo para el correo electrónico.
- Contraseña, es la contraseña que se requiere para autenticarse al hacer login.
- Rol, en este caso es un enumerador para saber que tipo es el usuario si es admin y cuenta con más privilegios o es un usuario sin privilegios de administrador.
- Hogar_id, clave foránea para poder relacionar al usuario con el hogar y sacar la información del hogar.
- Created_at, son los timestamps que se definen en la migración. En este caso cuando se crea un registro.
- Updated_at, son los timestamps que se definen en la migración. En este caso cuando se edita el registro.

NOTIFICACIÓN

- Id, se trata de la clave primaria.
- Tipo, campo para definir los diferentes tipos de notificaciones.

- Mensaje, es el mensaje que se guarda para mandar la notificación.
- Fecha_envío, la fecha en que se envía la notificación y saber cuánto tiempo lleva sin ser leída.
- Leída, booleano para saber si se ha leído o no y que salga el aviso.
- Usuario_id, clave foránea para relacionar y sacar los datos del usuario.
- Created_at, son los timestamps que se definen en la migración. En este caso cuando se crea un registro.
- Updated_at, son los timestamps que se definen en la migración. En este caso cuando se edita el registro.

TAREA

- Id, se trata de la clave primaria.
- Título, campo para el título de la tarea.
- Descripción, campo de tipo texto para escribir la descripción de la tarea.
- Estado, campo de tipo enum para saber en qué estado actual se encuentra la tarea.
- Prioridad, campo de tipo enum para saber cuan importante es una tarea.
- Fecha_creacion, campo de fecha para saber cuando se ha creado.
- Fecha_límite, campo de fecha para establecer cuál es la fecha límite de una tarea.
- Hogar_id, clave foránea para las relaciones con la tabla hogar.
- Usuario_id, clave foránea para poder relacionar los datos con la tabla usuarios.
- Created_at, son los timestamps que se definen en la migración. En este caso cuando se crea un registro.
- Updated_at, son los timestamps que se definen en la migración. En este caso cuando se edita el registro.

PRODUCTO

- Id, se trata de la clave primaria.
- Nombre, campo para escribir el nombre del producto.
- Categoría, tipo de categoría a la que pertenece el producto, no se ha hecho enum porque no se tenía totalmente claro si iba a haber un número definido de categorías o no.
- Descripción, campo de texto para establecer los detalles del producto.
- Unidad_medida, el tipo de medida que tiene el producto, es decir si se mide en litros, gramos, etc. Para saber la cantidad de este.
- Codigo_barras, el campo que recopila el código que se va a escanear y que identifica un producto, ya que es único.
- Created_at, son los timestamps que se definen en la migración. En este caso cuando se crea un registro.
- Updated_at, son los timestamps que se definen en la migración. En este caso cuando se edita el registro.

INVENTARIO

Esta tabla no se ha hecho uso en la aplicación ya que era redundante con la tabla de productos, como el proyecto ya estaba avanzado y no se podía establecer qué cambios y consecuencias habría en eliminar esta tabla se ha dejado en la base de datos.

- Id, se trata de la clave primaria.
- Cantidad_actual, de los productos que hay en un inventario. Este se trata de un campo redundante, por tanto en la aplicación del proyecto se dió cuenta de esto muy tarde, por eso no se ha quitado de la base de datos, pero no se ha utilizado.
- Stock, del producto en el inventario.
- Hogar_id, clave foránea para las relaciones con la tabla hogar.
- Producto_id, clave foránea para establecer las relaciones con la tabla producto.
- Created_at, son los timestamps que se definen en la migración. En este caso cuando se crea un registro.
- Updated_at, son los timestamps que se definen en la migración. En este caso cuando se edita el registro.

LISTA COMPRA

- Id, se trata de la clave primaria.
- Nombre, es el nombre de la lista de la compra.
- Fecha_creación, campo donde se recopila la fecha en la que se creó el registro.
- Estado, campo de tipo enum para saber si la lista ya se han comprado los productos o no, este campo es de cara al futuro por si se quiere tener un historial de listas de compras, para comparar precios, añadiendo el campo precio a producto.
- Hogar_id, clave foránea para las relaciones con la tabla hogar.
- Created_at, son los timestamps que se definen en la migración. En este caso cuando se crea un registro.
- Updated_at, son los timestamps que se definen en la migración. En este caso cuando se edita el registro.

LINEA LISTA COMPRAS

- Id, se trata de la clave primaria.
- Cantidad, cantidad de productos que se añaden a la lista de la compra.
- Lista_compra_id, clave foránea para obtener los datos de la lista de la compra.
- Producto_id, clave foránea para establecer las relaciones con la tabla producto.
- Created_at, son los timestamps que se definen en la migración. En este caso cuando se crea un registro.

- Updated_at, son los timestamps que se definen en la migración. En este caso cuando se edita el registro.

CALENDARIO

- Id, se trata de la clave primaria.
- Nombre, campo de tipo texto del nombre del calendario.
- Descripción, campo de tipo texto que guarda los detalles del calendario.
- Fecha_creación, fecha en la que se creó el registro.
- Created_at, son los timestamps que se definen en la migración. En este caso cuando se crea un registro.
- Updated_at, son los timestamps que se definen en la migración. En este caso cuando se edita el registro.

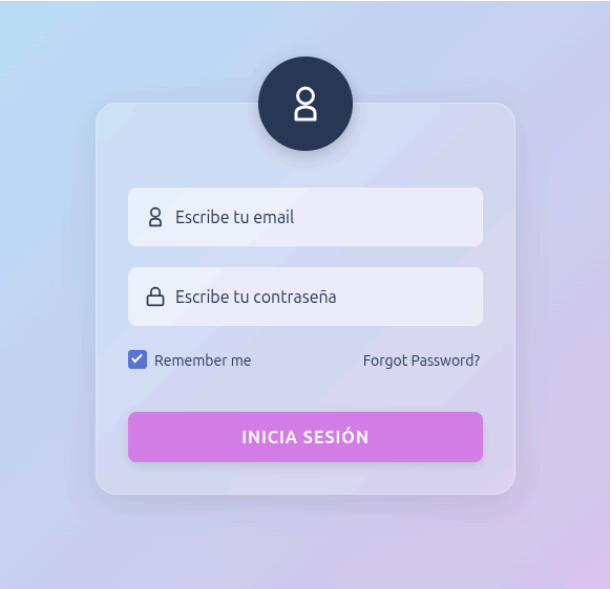
EVENTO

- Id, se trata de la clave primaria.
- Título, campo donde se va a escribir el título del evento.
- Fecha_inicio, campo que registra la fecha de inicio del evento.
- Fecha_fin, campo que registra la fecha final del evento.
- Tipo, campo de tipo enum que registra el tipo de evento que se va a crear.
- Calendario_id, clave foránea que obtiene los datos de la tabla calendario.
- Created_at, son los timestamps que se definen en la migración. En este caso cuando se crea un registro.
- Updated_at, son los timestamps que se definen en la migración. En este caso cuando se edita el registro.

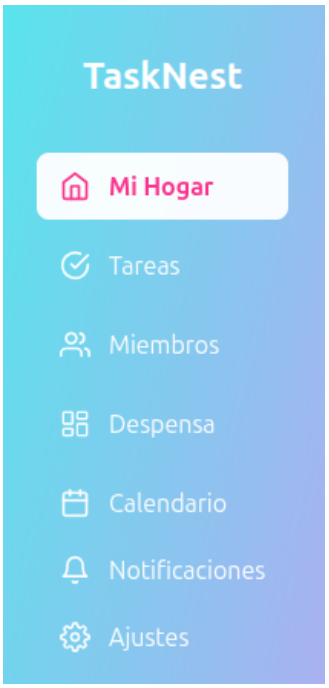
3.5. Funcionalidades de la aplicación.

En cuanto a las funcionalidades de la aplicación esta tiene distintas funcionalidades según la vista en la que se esté ubicado.

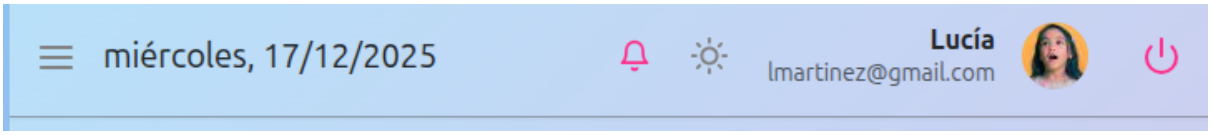
En primer lugar se encuentra la vista de Login que es donde los usuarios pueden poner sus credenciales y entrar a la aplicación.



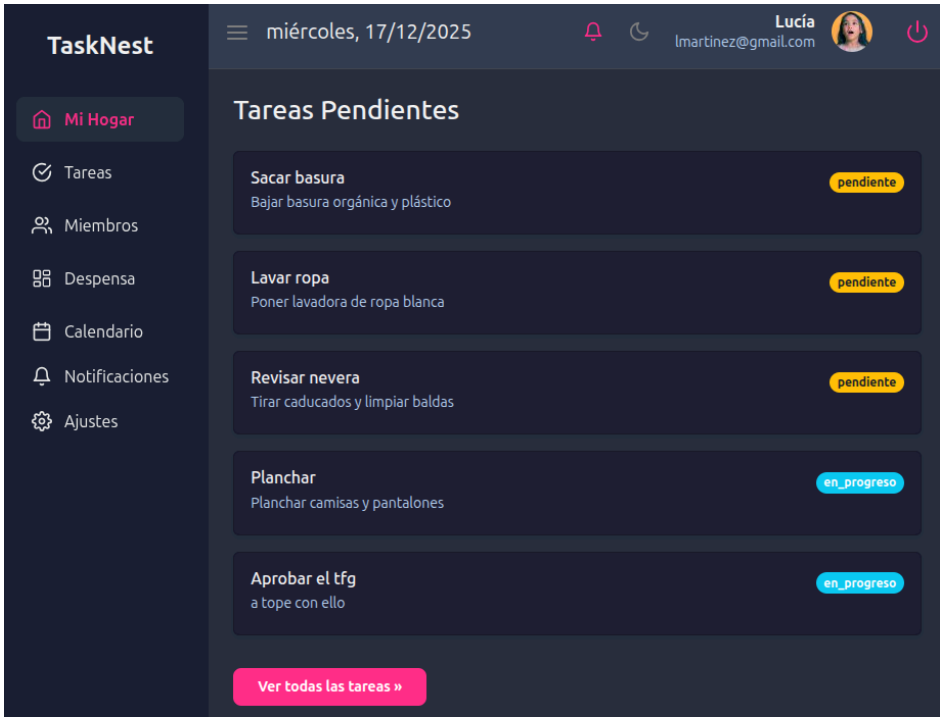
Una vez se entra a la aplicación se encuentra la vista Dashboard, está contiene varias funcionalidades y componentes que son comunes en todas las vistas. Por un lado a la izquierda de la aplicación está ubicado el componente del Sidebar el cual muestra una lista de enlaces a las distintas vistas que hay en la aplicación para que sea más fácil navegar entre ellas. Además de cambiar el diseño para que se sepa en qué vista nos encontramos.



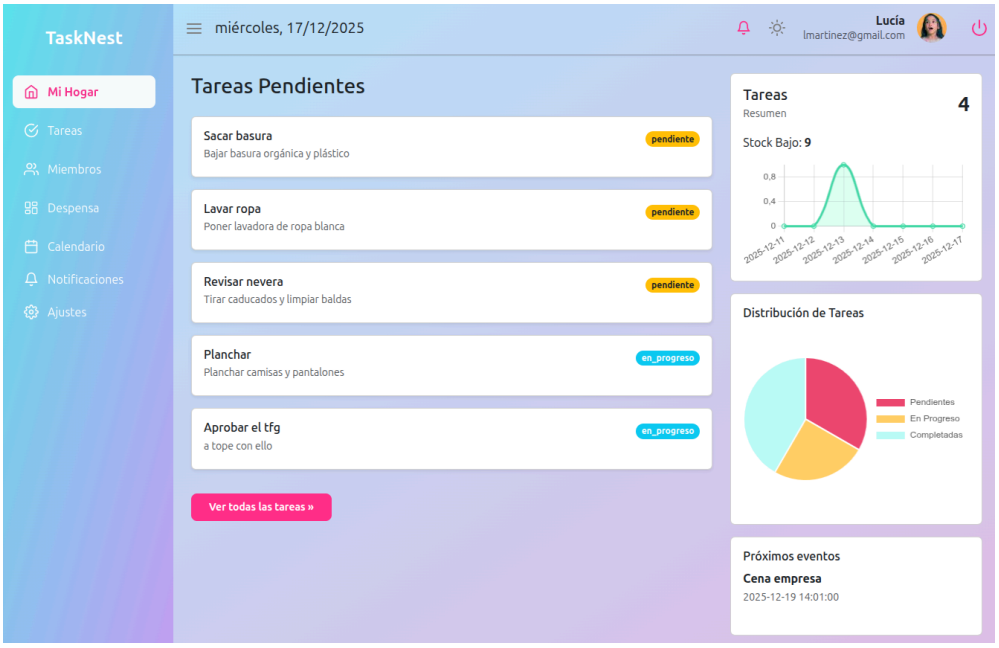
Por otro lado, el otro componente común en todas las vistas es el Topbar, este incluye varias funcionalidades. A la izquierda se tiene un botón hamburguer que desplaza escondiendo el sidebar o lo muestra si está escondido.



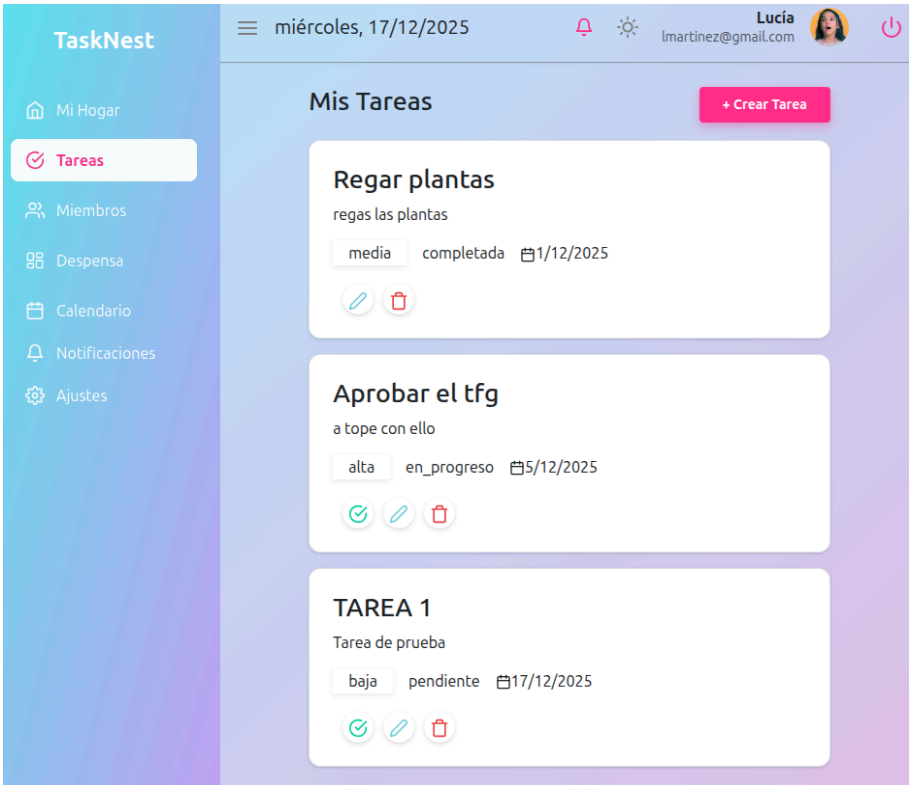
A su lado, la fecha actual, y luego tenemos dos funcionalidades que son bastante interesantes, por una parte tenemos la funcionalidad del modo oscuro, el cuál aplica este modo a toda la página en la que nos encontramos gracias a implementar una clase de estilos base, donde se declaran las variables necesarias para utilizar en el modo oscuro. Por otra parte, también se encuentra el icono de notificaciones que mostrará si el usuario tiene notificaciones pendientes por leer. En la parte más a la derecha de este componente se encuentra el usuario identificado junto a su icono de avatar, con la foto preestablecida. Otro aspecto a destacar es la funcionalidad de las notificaciones, estás se activan según del tipo que son es decir si son un evento, una tarea o forman parte de inventario.



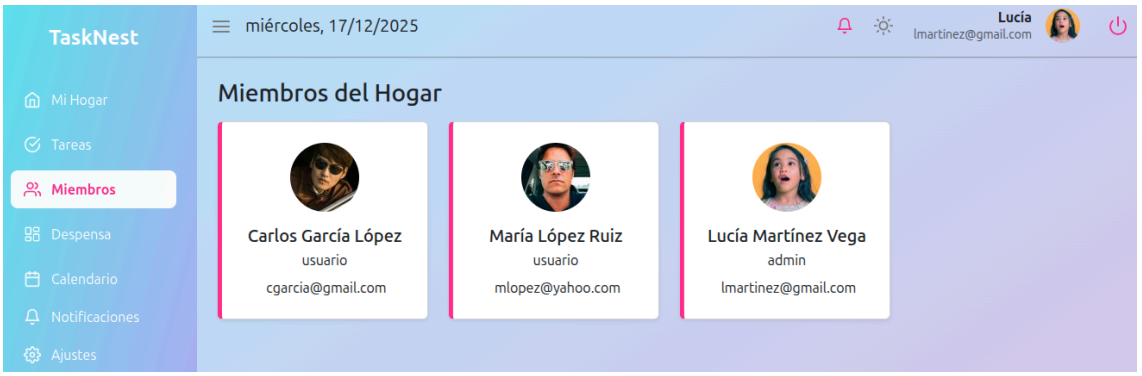
En la vista Dashboard, es donde se encuentra un resumen general de las tareas del hogar, no solo del usuario identificado, ya que está vista es común para todos los usuarios que formen parte de ese hogar. Además de esto, incluye gráficas de cuantas tareas se han creado y en cuanto tiempo, así como una estadística circular de qué tipo de tareas hay creadas, según el estatus en el que se encuentran.



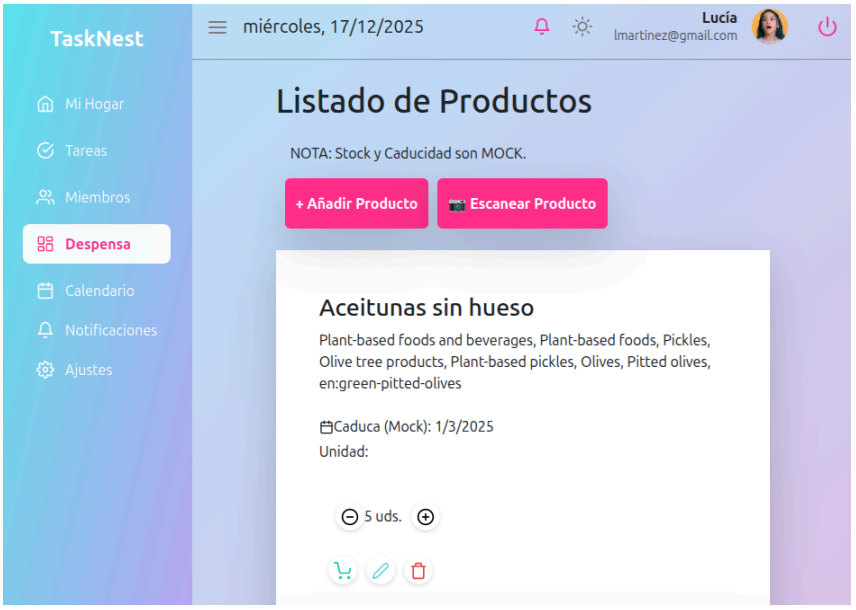
La siguiente vista que se encuentra en la aplicación es la vista de tareas, la cual incluye las tareas creadas por el usuario solamente, con una tarjeta que contiene los datos de la tarea creada, además será el mismo formulario crear que editar solamente que cuando se pulse el botón de crear se abrirá un modal para poder introducir los datos, y editar se editará sobre la propia tarea. Así como eliminar también tendrá un modal de tipo warning, que avisará al usuario si está seguro de eliminar la tarea. Esta funcionalidad es igual en productos solamente cambian los datos recogidos en el formulario. Además de esto tienen un campo de estado, para saber en qué estado se encuentra la tarea.



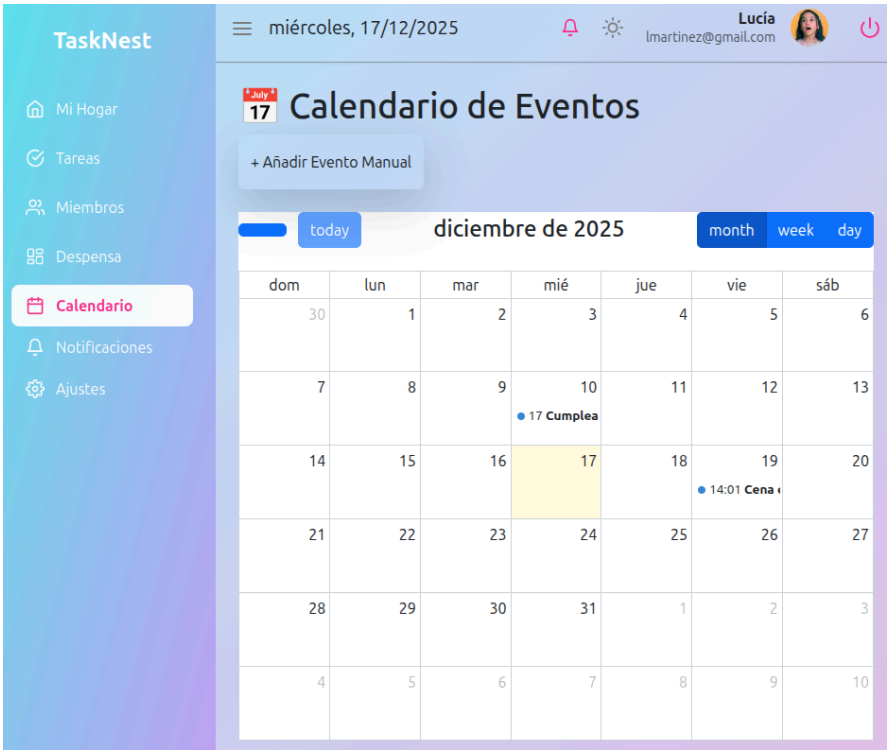
La siguiente vista es la vista miembros, donde se muestran los datos de los miembros del hogar que forma parte el usuario logueado. También incluye su rol si son de tipo administrador o un usuario común.



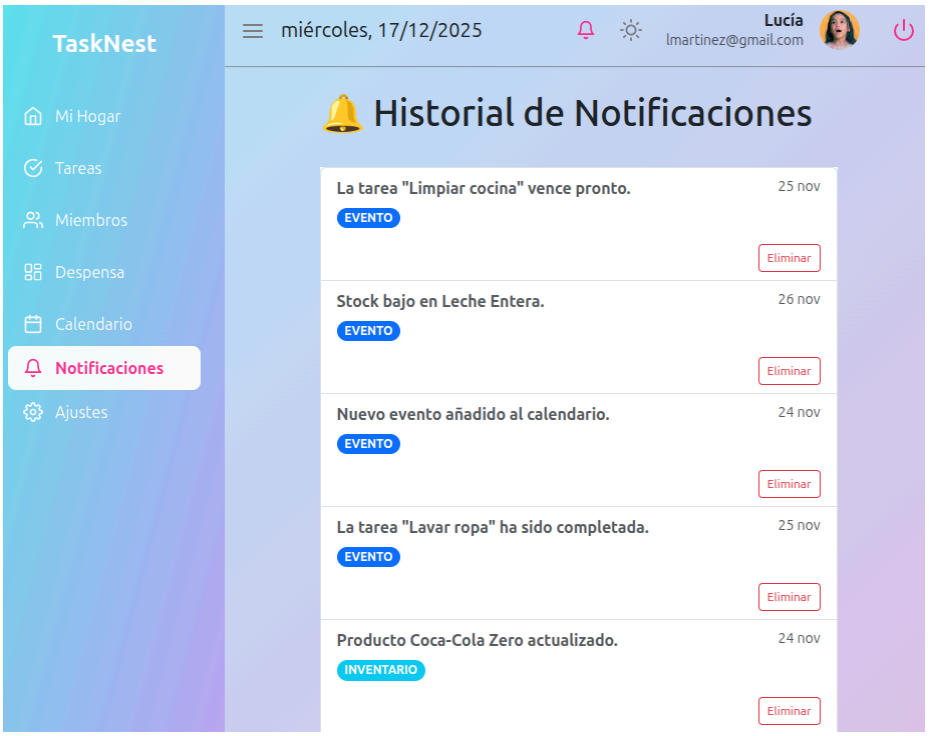
Al igual que la vista de tareas, productos tendrá la mismas funcionalidades, añadiendo dos funcionalidades más como es el botón de escanear producto donde se podrá introducir el código de barras del producto y la api open food facts devolverá los datos del producto a crear. En cuanto a la otra funcionalidad se trata de la lista de la compra, la cuál se crea sin ningún producto y en cada producto hay un botón de un carrito para añadir a la lista de la compra, si se añade dos veces el mismo producto aumenta el número de unidades del producto a comprar, también tiene un botón de eliminar el producto de la lista de la compra, lo que permite quitar productos que ya hayan sido comprados o que no hacía falta comprar.



Otra de las vistas más interesantes es esta vista ya que utiliza la librería de FullCalendar para crear el calendario, al hacer click en un día se podrá crear un evento nuevo, una vez creado el evento se podrá editar o eliminar haciendo click sobre el, ya que nos aparecerá la descripción de este, además de un botón de editar y otro de eliminar.



Por último, pero no menos importante tenemos la vista de las notificaciones que se diferencia del icono del componente del topbar en qué esta muestra un histórico de todas las notificaciones del usuario.



3.6. Recursos utilizados.

En este punto se van a listar los recursos que se han utilizado para poder realizar la aplicación web.

SO

Linux Mint

Se ha utilizado el sistema operativo de Linux distribución Linux Mint, ya que es más fácil hacer uso de los comando artisan de Laravel a través de la terminal de Linux, ofreciendo más facilidad a la hora de crear los directorios y los permisos que van a tener.

SGBD

PhpMyAdmin

Es la herramienta que se ha utilizado como gestor de base de datos, aplicación que ha resultado necesaria para gestionar las migraciones. La base de datos que se ha creado se ha realizado con MySql, ya que tiene un uso más extendido que MariaDB.

IDE

Visual Studio Code 1.105.1

Se ha utilizado como entorno de desarrollo y editor de código, está aplicación ya que permite un flujo rápido de trabajo, así como una organización visual y agradable para poder trabajar en ella. Otro aspecto a destacar, es el amplio abanico de extensiones que existen gracias a la comunidad que las crea.

Control de Versiones

Se ha utilizado Git a través de Github, es decir se han conectado la rama local develop con la rama remota develop, para tener un registro de los diferentes commits. No se han realizado issues ni pull request ya que se ha trabajado de forma individual, pero en el caso de haber trabajado en equipo si se hubiera tenido que organizar el trabajo con más detalle. No obstante, se ha realizado una normalización del tipo de commits que se hacían:

[new] → cuando se trata de un apartado completamente nuevo como un modelo, un controlador, etc.

[add] → cuando se ha añadido una nueva funcionalidad o sección a un bloque de código que ya no es de nueva creación.

[mod] → modificación de un apartado específico para mejorar la funcionalidad, no es que se añada código nuevo si no que se mejora la funcionalidad que tenía.

[fix] → son los arreglos de errores encontrados en el código.

Lenguajes de Programación y FrameWorks

En Lenguaje de servidor, se ha utilizado Php 8.3.6. utilizado conjuntamente con Laravel 11, el framework por excelencia de este lenguaje de programación.

En Lenguaje de cliente, se ha utilizado JavaScript (ES6+), desarrollando el frontend junto al framework de Vue 3.

Contenerización y despliegue

Se ha hecho uso de Docker para aislar y unificar los diferentes entornos de desarrollo junto a sus contenedores. Para poder llevar a cabo ese aislamiento, se ha empleado el uso de la herramienta docker compose.

Comunicación Cliente-Servidor

Se ha utilizado la librería de JavaScript Axios, para realizar las peticiones HTTP desde Vue.js hacía la API REST desarrollada en Laravel.

Bibliotecas y FrameWorks

- **Bootstrap,** se ha utilizado como librería para amenizar la realización de los estilos de la aplicación web.

- **Tailwind Css,** en casos concretos al igual que se ha usado bootstrap, se ha usado esta librería de estilos para complementar los estilos de los formularios.

- **FullCalendar,** se trata de una librería gratuita para implementar las funcionalidades de un calendario completo, esta librería puede llegar a incluir funciones premium, pero en este caso se ha utilizado la versión gratuita, la diferencia con otras librerías de calendario es que está permite la utilidad de

funciones más especificas, como drag & drop de eventos, lo que permite su creación. En este proyecto no se ha llegado a utilizar esta funcionalidad.

- **i.pravatar.cc,** no se trata de una librería como tal, pero si se ha utilizado como recurso para crear los avatares de cada usuario de la aplicación, ya que se trata de un random de imágenes de avatar de personas.

- **Quagga2,** se trata de una librería que permite realizar la función de escáner dentro de una aplicación web, ya que cuando se lanza la petición abre el permiso para utilizar la cámara trasera del móvil, es un aspecto a destacar que está más orientada al uso de dispositivos móviles. En este proyecto ha sido fundamental para poder implementar la lógica de la función de escanear productos.

- **Open Food Facts API,** por último tenemos esta API que se complementa con la anterior librería, ya que se trata de una API que contiene información de productos de comida alrededor del mundo, se puede utilizar escaneando el código de barras del producto a través de quagga 2 o mediante la introducción del código de barras y comprobando si existe. Esta es la función que se ha podido llevar a cabo en el proyecto de la gestión de tareas de la casa.

4. Estudio del coste económico y organizativo.

En el supuesto caso de que se quisiera implantar este proyecto en una situación real y publicarlo en gestores de descargas de aplicaciones como google play store o app store de apple, así como obtener un dominio donde publicar la aplicación y un servidor en la nube, para gestionar una mayor afluencia de usuarios como AWS (Amazon Web Service). Hacer uso de un servidor en la nube nos facilita los gastos de gestión de la aplicación, sobre todo si es una aplicación escalable a la que van a acceder varios usuarios a la vez haciendo peticiones a la base de datos.

En el caso de que la aplicación web no tuviera una gran escalabilidad y solo fuera utilizada en un entorno web, se necesitaría contratar un servicio de hosting. Es decir, un servicio donde crear y mantener el dominio público al que se podrá acceder a la web de dicha aplicación. Una de las empresas más conocidas que ofrecen este servicio es IONOS.

Teniendo en cuenta todos los aspectos comentados anteriormente el coste económico desglosado por servicios contratados sería el siguiente:

Servicio Contratado	Precio	Utilidades
Servidor en la nube AWS servicio de Amazon EC2	El precio según se indica en la web es bajo demanda. No se especifica cuánto costaría el servicio.	Se pueden generar instancias de un servidor, además de desplegar la web en un servicio externo sin tener que utilizar una máquina propia.
Publicar la aplicación en Google Play Store	Se necesita crear una cuenta específica de desarrollador en Google, es un pago único de 25$.	Permite subir versiones beta de la aplicación, además de poder acceder a más usuarios.
Publicar la aplicación en App Store de Apple	El precio para subir una app, es contratar una suscripción anual de 99$ en el programa de Apple Developer Program. En el caso de que la app genere ingresos se cobra una comisión del 15% o 30% sobre las ventas.	Permite subir versiones beta de la aplicación, además de poder acceder a más usuarios. Así como también ofrecen herramientas para perfeccionar y amoldar el código a los dispositivos de Apple.
Hosting de un dominio público en IONOS	Dependiendo de la extensión del nombre del dominio. Tiene un coste diferente, en este caso se compraría el host de un dominio con extensión .com que tiene un coste de 1€ por año contratado.	Incluye comprobación a nivel global del dominio, así como también de un certificado SSL Wildcard gratis y una asistencia de 24h. Se ha elegido está extensión ya que abarca un grupo más amplio de clientes.

5. Comparación con la situación actual y soluciones alternativas.

Según la información recopilada y tras un análisis de las aplicaciones existentes que se encargan de tipo de gestión del hogar, cabe destacar que la mayoría de ellas se centran en las tareas que se pueden realizar por los usuarios, además que la mayoría de estas se centra en un aspecto más concreto de la funcionalidad de la aplicación, en vez de tener diversas funcionalidades para ayudar a los usuarios a gestionar el hogar de maner más amplia y con diferentes opciones. Aunque es cierto, que al centrarse en un ámbito específico del hogar ofrecen funcionalidades más detalladas como generar puntos a través de realizar tareas y ganar

recompensas, o incluir recetas y preparaciones semanales para no pensar que comida se va a hacer cada día, lo que resulta muy práctico en la actualidad, ya que en muchos hogares las personas progenitoras no tienen el tiempo suficiente para dedicar el tiempo en preparar un menú semanal de comidas.

La mejora que se ve implementada con TaskNest, es que está orientada a ser una aplicación más global y amplia que incluya todo tipo de funcionalidades. Ahora mismo no se aprecia tanto este aspecto, ya que como se ha hecho hincapié está aplicación es una versión inicial de lo que puede llegar a ofrecer a distintos usuarios con diferentes perfiles, ya que un hogar está compuesto por personas de distintos perfiles, como infantes, adolescentes, personas jóvenes, personas adultas o personas mayores. El objetivo de esta aplicación al unificar varias funcionalidades, es facilitar al usuario lo que puede hacer en una misma app, sin tener que descargar distintas aplicaciones que ocupen demasiado espacio en el dispositivo que se vaya a usar.

Otra de las funcionalidades que tiene TaskNest que no es tan común en este tipo de aplicaciones es la gestión del inventario del hogar añadiendo productos a través de un escaneo del código de barras.

6. Conclusiones.

6.1. Resultados obtenidos.

Los resultados obtenidos, como he reiterado en apartados anteriores, han sido más bien los de una aplicación web como versión inicial, es decir, que aún no está lista para lanzarse al mercado ni instalarse en dispositivos, ya que hay muchas funcionalidades por mejorar y adaptar. En este caso se ha priorizado que tanto el backend como el frontend estén interconectados, funcionando correctamente y haciendo posible el poder escanear productos con la librería de Open Food Facts, aunque es cierto que el escaneado por imagen aún no está totalmente adaptado y habría que mejorar su lógica. Aún así, para una versión inicial se ha obtenido el resultado esperado con las funcionalidades que se marcaron en la propuesta del proyecto.

6.2. Puntos pendientes.

Como se ha comentado en el apartado anterior, al tratarse de una versión inicial han quedado muchos puntos pendientes de ajustar dentro de la aplicación. Uno de los puntos pendientes más relevantes es poner a prueba el escáner de productos a través de la cámara de un dispositivo móvil o una tablet en un entorno real, como es por ejemplo un supermercado a la hora de añadir los productos al carrito de la compra. Además de esto, también ha faltado la realización de estadísticas del precio

de los productos si estos varían, como también de una media de lo que cuesta realizar la compra según el supermercado.

Otro punto pendiente importante para la aplicación ha sido la implementación de toast de errores y éxito a la hora de crear, eliminar y editar un registro.

En definitiva, es una aplicación que tiene muchos aspectos por pulir y mejorar para poder implementarla en un entorno real, funcionando y validando los errores que puedan llegar a ocurrir una vez desplegada la aplicación, habiéndose puesto en marcha.

6.3. Tiempo dedicado, dificultad del proyecto en general y problemas.

El tiempo invertido en este proyecto, han sido muchas horas que visualmente no se aprecian, ya que se ha dedicado la mayor parte del tiempo a configurar correctamente el proyecto y la realización de todas las clases, controladores, seeders y migraciones de la parte del backend, además de pensar bien cómo iba a funcionar la lógica de negocio. Como no se ha documentado el tiempo exacto invertido en este proyecto se ha calculado una estimación de 120 horas, ya que se ha tenido de tiempo para realizar el proyecto desde el mes de octubre hasta el mes de diciembre, sin contar ambos meses de forma completa. Han sido muchas horas de entender lo que se quería hacer, cómo hacerlo y llevarlo a cabo en el código. Esta cantidad de horas invertidas está estrechamente relacionada con la dificultad del proyecto, ya que en sí no se trata de una idea difícil de llevar a cabo con tiempo, pero con un tiempo más limitado es un proyecto difícil de gestionar el tiempo de cada proceso a desarrollar, contando con los errores que se van encontrando a la hora de configurar tokens y la conexión de la parte servidor con la parte cliente. No es un proyecto que presuma por su dificultad técnica, si no por su dificultad elaborativa y de tener en cuenta todos los detalles, funciones y clases necesarias para su correcto funcionamiento.

En cuanto a las problemáticas que han surgido durante el desarrollo, se ha centrado en un error de configuración que ha afectado intrínsecamente al desarrollo del login de la aplicación, es una de las funcionalidades más básicas e indispensables que ha de tener una aplicación web en la actualidad, en un principio está iba a funcionar con laravel sanctum a través de tokens de sesión, pero al final se ha realizado un login sin usar este tipo de autenticación, ya que no se ha podido esclarecer el error de que el frontend no recogía el token enviado por el backend, aunque este fuera correcto. Se ha revisado la configuración y los headers de nginx y todo era correcto, salvo el hecho de que al frontend no le llegaban los datos de inicio de sesión, en este error se han invertido 2 semanas de trabajo y debido al poco tiempo se decidió seguir avanzando sin esta funcionalidad acabada ni corregida. Es una de las partes más frustrantes de la realización del proyecto, que quedan pendientes a la hora de

perfeccionar la aplicación. Se ha llegado a la conclusión de que una de las posibles causas de este error sea por trabajar en localhost, aunque no se ha resuelto la duda ya que hay mucha documentación ambigua sobre este error en el caso de esta aplicación.

6.4. Valoración personal y auto evaluación.

Por un lado, considero que es un proyecto muy completo, ya que incorpora todo lo visto durante el ciclo superior, es decir parte de frontend, parte de backend como API y una base de datos sólida, todo esto contenerizado en una plataforma como es docker para mantener la aplicación dividida en contenedores y que facilita la puesta en marcha de esta. Aun así, es cierto que hay tablas en la base de datos que han quedado redundantes y se podría haber tenido en cuenta, ya que esto ha acabado afectando al desarrollo de la parte del cliente de la aplicación.

En cuanto al desarrollo y el código utilizado, hay métodos y partes de este que se podrían optimizar para que el funcionamiento de la aplicación fuera más sencillo de entender y rápido de compilar. Aun así, se ha conseguido llegar al objetivo siendo una aplicación funcional. Sí que es verdad que queda mucho por pulir y no es una aplicación preparada para presentarse ante un cliente como modelo final, pero sí como una demo, ya que tiene todo lo necesario para entender qué funcionalidades tiene y cómo se van a mostrar los datos. A parte de esto, otro aspecto que no se ha tenido en cuenta a la hora de realizar un proyecto de esta índole es la realización de pruebas y test para la comprobación de que los datos recogidos y guardados son correctos, así como las pertinentes validaciones. Son cosas pendientes que se tienen que realizar con más tiempo y una organización más detallada, así como invertir parte de ese tiempo en documentar el proyecto, para saber las funcionalidades de cada vista/apartado de este.

Por otro lado, en cuanto a mi autoevaluación siento que estoy orgullosa de la aplicación que he hecho, ya que es la primera vez qué trabajo en un proyecto con tanta envergadura y ambición. Al principio, fue una idea para satisfacer una necesidad personal pero que ha acabado siendo como un viaje hacia lo desconocido de una parte de mí misma que no conocía, ya que al pasar tantas horas en la realización de este, sin pautas ni ejercicios, he conseguido disfrutarlo, aún en aquellos momentos de frustración extrema de tener ganas de mandar el proyecto por la borda, pero gracias a toda la experiencia durante las horas desarrollandolo como también en las prácticas, puedo decir que cada vez me gusta más desarrollar aplicaciones, funcionalidades, botones y centrar divs. Dicho todo esto, sé que me queda mucho por aprender y por mejorar mi lógica de programación, ya que es una de las cosas en las que tengo más carencias. Pero no todo puede ser malo, ya que aunque es cierto que desarrollar me lleva más tiempo, el diseño de la web no tanto, es uno de los apartados que me gusta retocar y dedicar tiempo, ya que considero importante la presentación de una aplicación de

cara al uso que va a tener y la experiencia que van a tener los usuarios a la hora de utilizarla. Es por eso, que si que me siento un poco decepcionada, ya que no he podido dedicar todo el tiempo que me hubiera gustado a arreglar el diseño de algunos apartados, aún así estoy muy satisfecha del trabajo que he realizado.

6.5. Consejos más destacables.

Uno de los consejos más importantes a la hora de plantear un proyecto de tal envergadura, es la importancia que toma la organización y distribución del tiempo que se tiene. Así como saber aceptar que hay ciertos momentos en los que vale la pena tomarse un descanso y continuar en otro momento, porque muchas veces desconectando un poco vemos otro punto de vista u otra solución que no estábamos teniendo en cuenta.

En conclusión, saber gestionar el tiempo y las emociones es lo que hace que una aplicación pueda salir adelante, y dejar de lado ciertos aspectos que no sabemos solucionar para retomarlos con más calma en otro momento.

7. Necesidades y sugerencias de formación.

Durante la formación del ciclo superior de aplicaciones web, tanto los conocimientos aprendidos en el primer curso, como los conocimientos aprendidos en el segundo curso, han sido un pilar fundamental para el desarrollo de está aplicación web. Sobre todo, en la parte del backend y la configuración de docker que puede ser uno de los aspectos más problemáticos de la creación de un proyecto totalmente nuevo.

Aún así, se han ido manifestando ciertos conocimientos que han sido necesarios para poder realizar este proyecto como la realización de proyectos más innovadores y adaptados a las necesidades actuales de las empresas, ya que en el ámbito de la tecnología está evoluciona a cada mes de manera impredecible, es por eso que no siento que se requiera una necesidad en cuanto a la base de los conocimientos que ofrece el superior de desarrollo de aplicaciones web, sino más bien una necesidad de incorporar nuevas tecnologías en los proyectos realizados en el curso, como la inclusión de la Inteligencia Artificial como funcionalidad dentro de un proyecto, o adentrarse en la exploración de migrar una aplicación web a una aplicación móvil, ya que hoy en día es casi un requisito que una aplicación pueda ser accesible desde un dispositivo móvil y se adapte a este tipo de dispositivos, sobre todo en dispositivos como las tablets.

En definitiva, las sugerencias de formación se basan en una propuesta de realizar proyectos adaptados a las necesidades actuales, ya que el objetivo final es poder desarrollar aplicaciones web ya sea por cuenta ajena o por cuenta propia, es decir, programando para una empresa o programando de forma autónoma. Otro aspecto a destacar, son las cuestiones de seguridad y brechas que pueden haber a la hora de

desarrollar formularios, ya que la gran mayoría de aplicaciones web contienen formularios. No es que no se dé importancia a estos conocimientos, pero sí que es verdad que por falta de tiempo no se adentra más en ellos, cuando es un aspecto muy importante del desarrollo web.

8. Bibliografia

- Amazon AWS EC2, información sobre la contratación del servicio, extraída de:
  https://aws.amazon.com/es/ec2/pricing/on-demand/

- Google Play Store, información sobre cómo publicar una app, extraída de la fuente:
  https://play.google.com/intl/es/console/about/guides/releasewithconfidence/

- App store de Apple, información sobre cómo publicar una app, extraída de la fuente:
  https://www.apple.com/es/app-store/developing-for-the-app-store/

- IONOS, información sobre la contratación de un dominio, extraída de la fuente:
  https://www.ionos.es/dominios/com-dominio

- Educación 3.0. *20 apps gratuitas para organizar las tareas domésticas en familia*. Información extraída de la siguiente fuente:
  https://www.educaciontrespuntocero.com/familias/apps-gratuitas-organizar-tareas-domesticas-familia/

- Chat Gpt (OpenAI), se ha utilizado para consultar dudas de errores de código y optimización de este.
  https://chatgpt.com/

- Github, usado para el control de versiones y seguimiento del desarrollo.
  https://github.com/

- Git, Se ha utilizado de manera conjunta con github para el control de versiones.
  https://git-scm.com/docs

- Visual Studio Code, se ha utilizado como editor de código.
  https://code.visualstudio.com/

- Laravel, se ha utilizado la documentación de laravel tanto para la realización de las migraciones, lenguaje de Eloquent como dudas y errores surgidos.
  https://laravel.com/docs/12.x

- Laracasts, se ha utilizado para dudas y errores en la implementación de autenticaciones mediante CORS y Sanctum.
  https://laracasts.com/discuss

- Vue, se ha utilizado la guía de instalación de vue para saber que extensiones requería él proyecto como por ejemplos axios y router.
  https://es.vuejs.org/v2/guide/

- Bootstrap, documentación oficial sobre las clases y colores.
  - https://getbootstrap.com/docs/5.3/getting-started/introduction/
  - https://getbootstrap.com/docs/4.0/utilities/colors/

- Tailwind Css, documentación oficial sobre su uso.
  https://tailwindcss.com/docs/installation/framework-guides